Coursework 2 Component 1: Topic Proposal

1. Introduction

Customer churn is one of the challenges that large companies face due to its effect on the revenue, especially for the company gaining revenue from user subscription such as telecom company and stream media company. Hence, in this coursework, we want to build a prediction model to predict the likelihood of churning for music streaming app user by using the datasets from Sparkify. Sparkify is a virtual music streaming service created by Udacity whereby user can enjoy music without fee by watching the advertisement in between songs or subscribe for a premium subscription for a fee. The users are allowed to upgrade, downgrade or cancel the subscription at any time. Generally, it takes lesser cost to keep and sustain the existing user than acquire new user. Therefore, we like to predict the user which is likely to churn in order to provide targeted offers to these users before they churn and cancel the cancel the subscription.

1.1 Project Goal

The project goal is to create a binary classifier based on the datasets to analyze the activities of the user with Sparkify virtual services and define the user that potential to cancel the service and churn. We aim to study the user churn from the big data of Sparkify user logs and apply machine learning technique to create an optimum predictive model on user churn. The study also helps us to understand the application of PySpark and MLLib. Eventually, our main objective is to compare a few predictive models and obtain the accuracy of each model to deploy the best model on user based churning prediction through machine learning approach.

1.2 Data Collection and Summary

Due to the size of the datasets which is relatively large, we required to work on the data with PySpark, Spark ML and Apache Hadoop. The path to the studied data on HDFS is 'hdfs://www.dlmd.com:9000/input/mini_sparkify_event_data.json'. In this coursework, we will be using the subset of the 12gb data which contain 286,500 rows and 18 columns of data.

The data file is in JSON format, and it was read by PySpark dataframe for the analysis and model prediction. By running jsondataframe.printSchema(), we are able to contain the features of the datasets. The features from the user log included artist name, authentication method, user first name, user gender, user last name, length of the song that user listened, user subscription level either paid or free, user location, service method, service interaction page, user registration timestamp, session id, song name by user, status, timestamp of user service, web browser used and the user id. These features can be categorized as user information, Sparkify service information, user service event and artist information.

```
In [4]: jsondataframe.printSchema()

root
 |-- artist: string (nullable = true)
 |-- auth: string (nullable = true)
 |-- firstName: string (nullable = true)
 |-- gender: string (nullable = true)
 |-- itemInSession: long (nullable = true)
 |-- lastName: string (nullable = true)
 |-- length: double (nullable = true)
 |-- level: string (nullable = true)
 |-- location: string (nullable = true)
 |-- method: string (nullable = true)
 |-- page: string (nullable = true)
 |-- registration: long (nullable = true)
 |-- sessionId: long (nullable = true)
 |-- song: string (nullable = true)
 |-- status: long (nullable = true)
 |-- ts: long (nullable = true)
 |-- userAgent: string (nullable = true)
 |-- userId: string (nullable = true)
```

1.3 Method-Data Pre-processing and Cleaning

To begin with the analysis and construct the predictive model, we will import all necessary classification and evaluation libraries from pyspark.sql and pyspark.ml. Then, a Spark session is built.

```
# import libraries
from pyspark.sql import SparkSession
from pyspark.sql.functions import avg, col, concat, desc, explode, lit, min, max, split, udf,regexp_extract,when
from pyspark.sql.types import IntegerType,DateType
from pyspark.ml import Pipeline
from pyspark.ml.classification import LogisticRegression
from pyspark.ml.evaluation import MulticlassClassificationEvaluator
from pyspark.ml.classification import LogisticRegression, GBTClassifier
from pyspark.ml.feature import StandardScaler,OneHotEncoder,CountVectorizer, IDF, Normalizer, RegexTokenizer, StandardScaler, Stc
from pyspark.ml.tuning import CrossValidator, ParamGridBuilder
from pyspark.ml.classification import DecisionTreeClassifier, RandomForestClassifier
from pyspark.sql import functions as F
import seaborn as sns
import matplotlib.pyplot as plt
import datetime
import re
import pandas as pd
from pyspark.sql import Window as window
import warnings
```

```
#ignore warning
warnings.filterwarnings('ignore')
#building spark
spark = SparkSession.builder \
    .appName("musicPredict") \
.getOrCreate()
```

Upon importing the libraries, the data is loaded from the HDFS path followed by data cleaning and pre-processing. The table below shows the preview of Sparkify user interaction log the from the top five row.

```
#preview top 5 row of data
music_data_preview[:5]
```

	artist	auth	firstName	gender	itemInSession	lastName	length	level	location	method	page	registration	sessionId	song	status	
0	Martha Tilston	Logged In	Colin	M	50	Freeman	277.89016	paid	Bakersfield, CA	PUT	NextSong	1.538173e+12	29	Rockpools	200	1
1	Five Iron Frenzy	Logged In	Micah	M	79	Long	236.09424	free	Boston-Cambridge-Newton, MA-NH	PUT	NextSong	1.538332e+12	8	Canada	200	1
2	Adam Lambert	Logged In	Colin	M	51	Freeman	282.82730	paid	Bakersfield, CA	PUT	NextSong	1.538173e+12	29	Time For Miracles	200	1
3	Enigma	Logged In	Micah	M	80	Long	262.71302	free	Boston-Cambridge-Newton, MA-NH	PUT	NextSong	1.538332e+12	8	Knocking On Forbidden Doors	200	1
4	Daft Punk	Logged In	Colin	M	52	Freeman	223.60771	paid	Bakersfield, CA	PUT	NextSong	1.538173e+12	29	Harder Better Faster Stronger	200	1

In data cleaning, we first remove data with null or NA records in userID field as null record will affect the outcome of churn prediction. Besides, we also remove any duplicated userID from the data and we observe that there is 225 distinct user ID in this subset of datasets.

```
#remove null userId
jsondataframe=jsondataframe.filter(jsondataframe["userId"] != "")
print("The number of records filtered by userid is as follows: ",jsondataframe.count())
```

```
The number of records filtered by userid is as follows:  278154
```

```
#remove the duplicate userId
distinctUserid=jsondataframe.select("userId").dropDuplicates().count()
print("Number of different user ids:",distinctUserid)
```

```
Number of different user ids: 225
```

All the timestamp used in the datasets are in milliseconds unit of measure. Therefore, we converted the timestamp to seconds and run the timestamp by strftime() to convert the time to string format of Year-Month-Date, Hour-Minutes-Seconds.

```
#convert the timestamp to year:month:day hour:minute:seconds format, rename as BehaviorTime
convertTime = udf(lambda x:datetime.datetime.fromtimestamp(x/1000.0).strftime ( "%Y-%m-%d %H:%M:%S"))
jsondataframe=jsondataframe.withColumn("BehaviorTime",convertTime('ts'))
```

```
#retrieve hour data as hour based on the timestamp feature
get_hour = udf(lambda x: datetime.datetime.fromtimestamp(x / 1000.0).hour)
jsondataframe = jsondataframe.withColumn("hour",get_hour(jsondataframe.ts).cast('int'))
```

```
#retrieve week data as weekday based on the timestamp feature
get_week = udf(lambda x: datetime.datetime.fromtimestamp(x / 1000.0).weekday())
jsondataframe = jsondataframe.withColumn("weekday",get_week(jsondataframe.ts).cast('int'))
```

Apart from that, we also pre-process the important features such as user location, the device and browser that user used for streaming Sparkify.

```
#retrieve user state data based on the location feature
jsondataframe = jsondataframe.withColumn("location_state",regexp_extract("location",",(.*)",1))
```

```
#retreive user device based on the userAgent feature
device_function = F.udf(lambda x: x.split('(')[1].split(";")[0].split(" ")[0] .replace('"', ' ').strip() if not x is None else "
jsondataframe = jsondataframe.withColumn("device",device_function(jsondataframe.userAgent))
```

```
#retrieve user browser based on the userAgent feature
browser_function = F.udf(lambda x: x.rsplit(' ', 1)[1].rsplit('/', 1)[0] .replace('"', ' ', 1).strip() if not x is None else "")
jsondataframe = jsondataframe.withColumn("browser",browser_function(jsondataframe.userAgent))
```

The most critical feature for churn prediction model on streaming platform is the page feature which hold the interaction and event logs of users on Sparkify. Based on the preview code, we can retrieve all event logs from “page” and cancellation confirmation

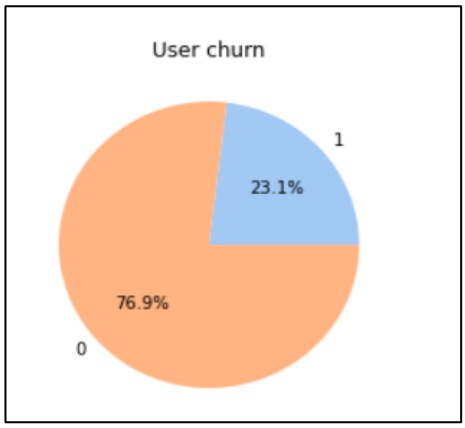
from the log indicating the churn of user. From this dataset, we observed 52 cancellation confirmation events.

```
#summarize the interaction and event logs from the page feature
music_data_preview['page'].value_counts()
```

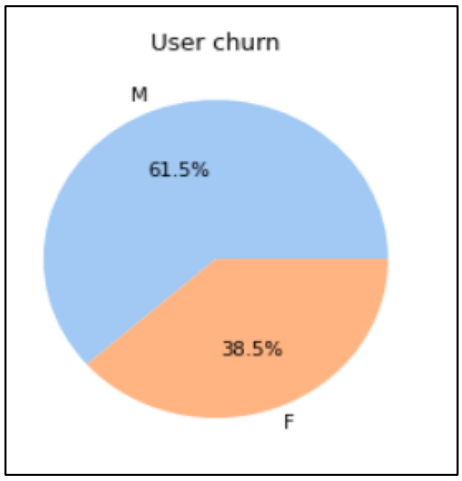
```
NextSong                    228108
Home                         14457
Thumbs Up                    12551
Add to Playlist               6526
Add Friend                    4277
Roll Advert                   3933
Login                         3241
Logout                        3226
Thumbs Down                   2546
Downgrade                     2055
Help                          1726
Settings                      1514
About                          924
Upgrade                        499
Save Settings                  310
Error                          258
Submit Upgrade                 159
Submit Downgrade                63
Cancel                          52
Cancellation Confirmation       52
Register                        18
Submit Registration              5
Name: page, dtype: int64
```

1.4 Method-Exploratory Data Analysis

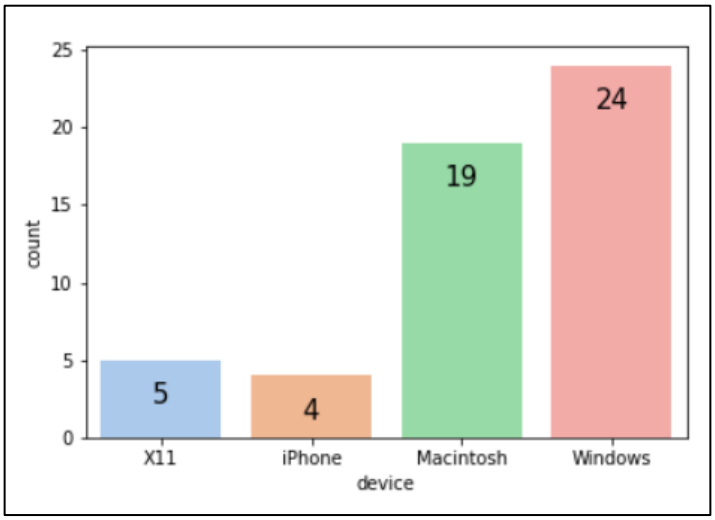
A function is created for cancellation confirmation to label the churn of user. From the event logs, user that hit on cancellation confirmation will be label as count 1 whereas it will be 0 for active user. To summarize, 23.1% of the user churn from Sparkify while 76.9% of users are active user,

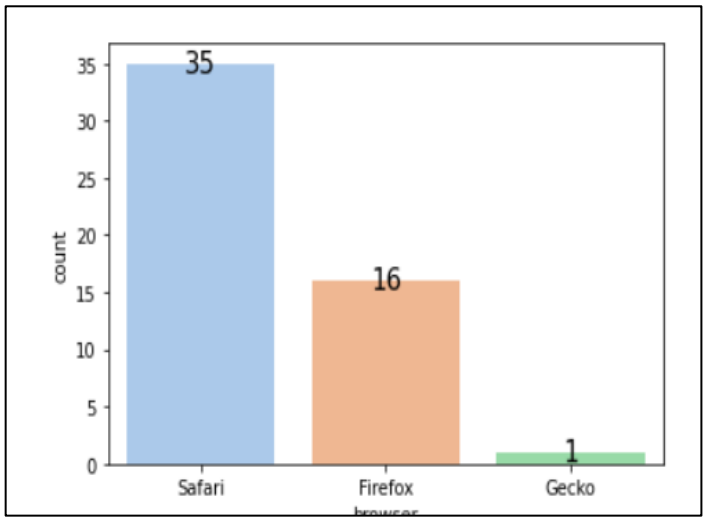


After determining the user churn percentage, we want to study and analysis the relationship between various features and the churn rate. Firstly, we analyze the user churn from gender view. Based on the results, the churn user consists of 61.5% male and 38.5% female.

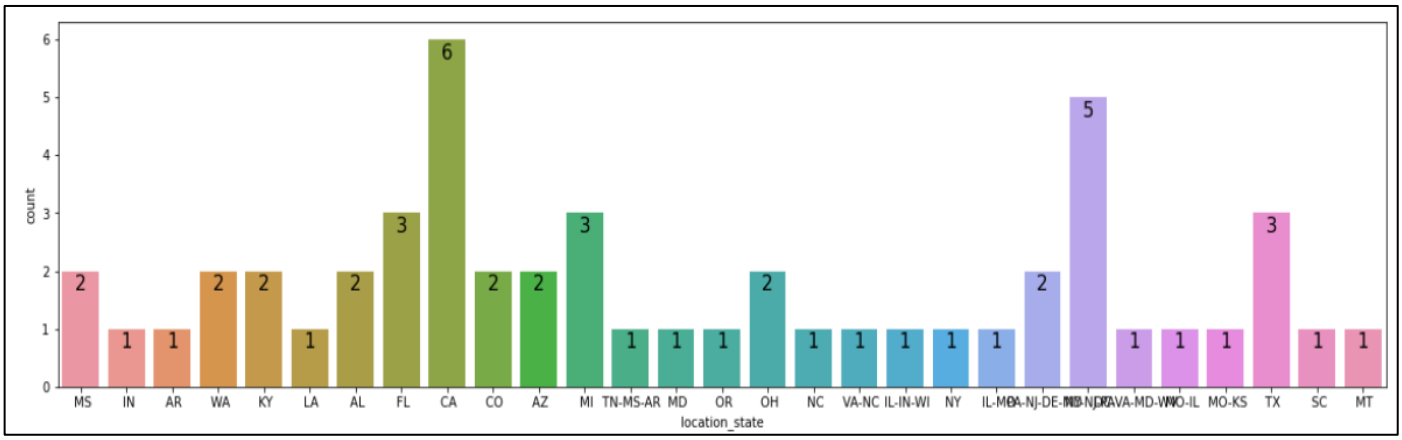


For the device used to stream Sparkify, 46.15% of user churn using Windows, 36.5% using Macintosh, 7.6% using iPhone while 9.6% using X11. As for the browser used for churn user, 67.3% using Safari, 30.77% using Firefox while 1.9% using Gecko. Therefore, we know that the highest number of device and browser used to stream Sparkify for churned users are Windows and Safari respectively.

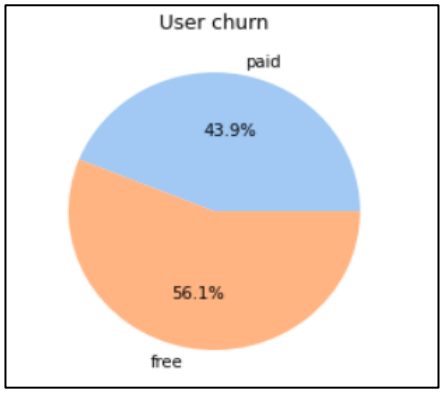




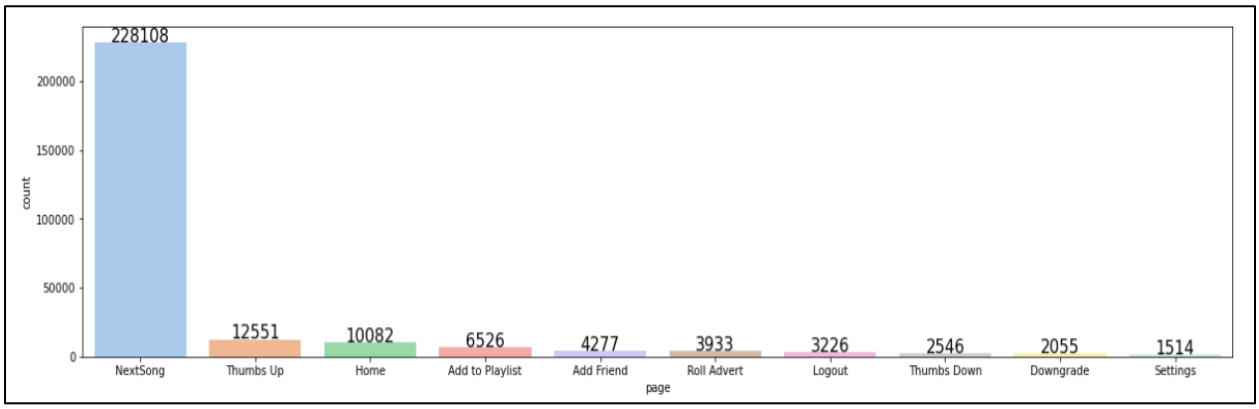
By working on the location feature, we observed that highest number of churned users are from California state.



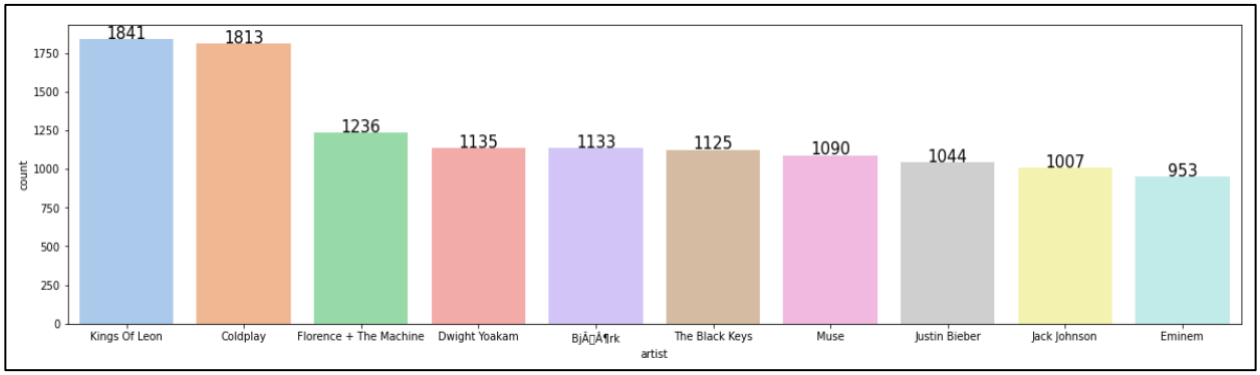
The free user usually churns as compared to paid user as 56.1% of churned user are user using the free services.



The user most visited page on the event logs is “next song” event whereby user is streaming the next music whereas the lowest event is “settings”



The user listened to King of Leon and Coldplay at the highest frequency on Sparkify.



1.5 Method-Feature Engineering

Upon completion of data cleaning and exploratory data analysis, we want to extract and analyze the feature which is meaningful for a prediction classification model from the log file. There are 17 features being engineered to determine the possible impact of the features on the training model.

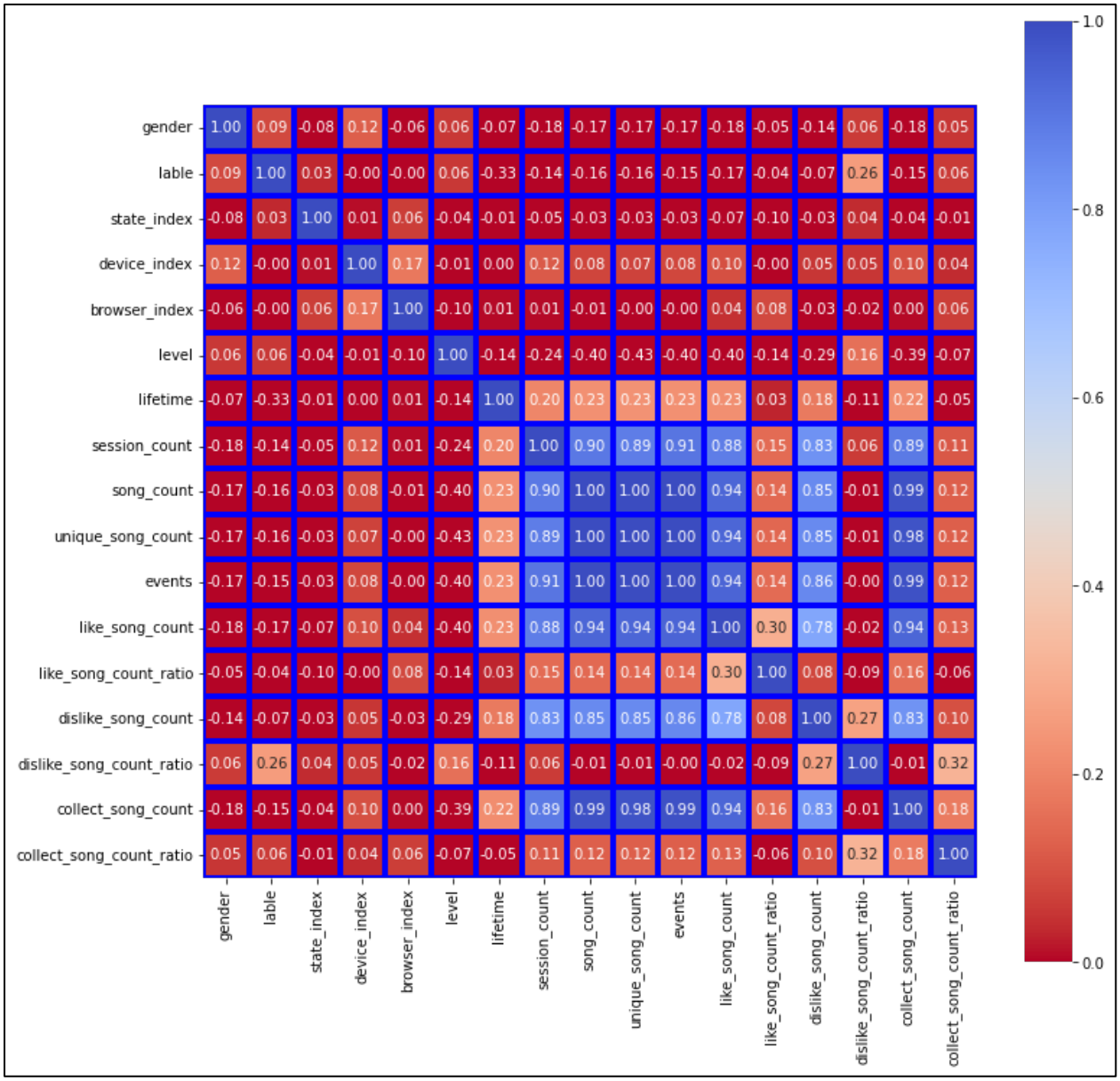
Features	Description
User ID	Unique user ID
Gender	User gender
Label	Churn label, 1 = churn, 0 = active
Location_state	User location
Device	User device
Browser	User browser
Level	User level (free or paid service)
Lifetime	User's time spent on Sparkify
Session_count	Number of user session

Unique_song_count	Number of different song user listened to
Like_song_count	Number of songs liked by user
Like_song_count_ratio	Ratio of liked songs
Dislike_song_count	Number of songs disliked by user
Dislike_song_count_ratio	Ratio of disliked songs
Collect_song_count	Number of songs favorited by user
Collect_song_count_ratio	Ratio of favorited songs

By converting the data features to Panda data frame, we vectorized the features by using string indexer and one hot encoder converter. One Hot Encoding is the process of transforming categorical variables into numerical format before fitting and training a Machine Learning algorithm while string indexer helps to convert categorical string columns in a Data Frame into numerical indices. By vectorization of all the features, we are able to fit the variables and analyzed the correlation of these features with the label count.

```
#Vectorization of features

#dfeine string indexer
StringIndexer_state = StringIndexer(inputCol='state', outputCol='state_index')
StringIndexer_device = StringIndexer(inputCol='device', outputCol='device_index')
StringIndexer_browser = StringIndexer(inputCol='browser', outputCol='browser_index')

#define one hot encoder
OneHotEncoder_state= OneHotEncoder(inputCol='state_index', outputCol='state_vector')
OneHotEncoder_device= OneHotEncoder(inputCol='device_index', outputCol='device_vector')
OneHotEncoder_browser= OneHotEncoder(inputCol='browser_index', outputCol='browser_vector')

#define pipeline and transform model
my_pipeline = Pipeline(stages=[StringIndexer_state, StringIndexer_device, StringIndexer_browser, OneHotEncoder_state,OneHotEncode
pipelineModel=my_pipeline.fit(jsondataframe)
jsondataframe =pipelineModel.transform(jsondataframe)

jsondataframe.toPandas()[:5]
```

A heatmap is produced to plot the correlation among all 17 features. The closer to value 1.0 (blue) in heatmap indicating the stronger is the correlation between features. However, in order to study and analyze the churn, we mainly focus on the correlation between all features with the label feature as the label indicating whether the user churn or active.



We sort out the correlation of all features with label by descending order from positive to negative value. Positive correlation indicating the variables are moving in tandem with label feature in the same direction whereas negative correlation indicating the variables is moving in opposite direction. The closer the correlation to either 1 or -1 indicating strong correlation whereas the correlation is weak if the value is closer to 0. Therefore, to build the predictive model, we will remove device_index and browser_index from the training as these features are close to 0 indicating their weak correlation with label count which represent user churn.

```
lable                        1.000000
dislike_song_count_ratio     0.255117
gender                       0.085340
collect_song_count_ratio     0.058951
level                        0.055309
state_index                  0.034935
device_index                -0.002921
browser_index               -0.004177
like_song_count_ratio       -0.044940
dislike_song_count          -0.074668
session_count               -0.142129
collect_song_count          -0.151875
events                      -0.154349
song_count                  -0.156110
unique_song_count           -0.159024
like_song_count             -0.168105
lifetime                    -0.328511
Name: lable, dtype: float64
```

After defining the important features that have strong correlation with user churn, we will finalize the features list for model training. VectorAssembler is used to gather the vectorized variables follow by running StandardScaler to standardize the variables value to standard format since the value for different features could have different range. For example, event value could be up to 1000 but ratio value can only be 0 to 1. After all the standardization, we create the features pipeline and ready to execute Machine Learning.

The dataset is split into two parts whereby 70% of random data is categorized as train data and 30% is categorized as test data.

```
#execute machine learning
jsondataframeML = features_pipeline.fit(jsondataframeML).transform(jsondataframeML)
jsondataframeML.select("features","lable").show()
```

```
#split datasets to 70% train data and 30% test data
train_dataset,test_dataset = jsondataframeML.randomSplit([0.6, 0.4], seed=100)
```

```
+--------------------+-----+
|            features|lable|
+--------------------+-----+
|(70,[2,3,4,5,6,7,...|    0|
|(70,[0,2,3,4,5,6,...|    0|
|(70,[2,3,4,5,6,7,...|    1|
|(70,[0,1,2,3,4,5,...|    1|
|(70,[0,1,2,3,4,5,...|    0|
|(70,[0,2,3,4,5,6,...|    0|
|(70,[2,3,4,5,6,7,...|    0|
|(70,[0,2,3,4,5,6,...|    1|
|(70,[0,2,3,4,5,6,...|    0|
|(70,[2,3,4,5,6,7,...|    0|
|(70,[0,1,2,3,4,5,...|    1|
|(70,[0,1,2,3,4,5,...|    1|
|(70,[1,2,3,4,5,6,...|    0|
|(70,[0,2,3,4,5,6,...|    0|
|(70,[0,2,3,4,5,6,...|    1|
|(70,[1,2,3,4,5,6,...|    0|
|(70,[0,2,3,4,5,6,...|    0|
|(70,[0,2,3,4,5,6,...|    0|
|(70,[0,2,3,4,5,6,...|    0|
|(70,[0,2,3,4,5,6,...|    1|
+--------------------+-----+
only showing top 20 rows
```

1.6 Method- Model Training

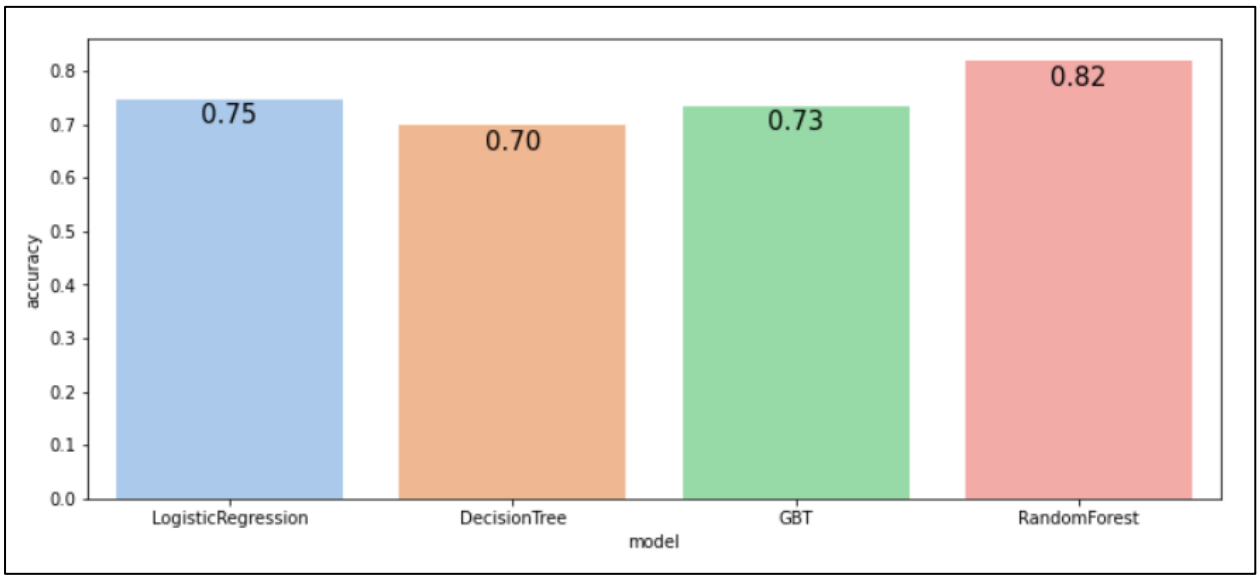
By splitting the dataset to train data and test data, we proceeded with model creation and training. In this coursework, we will apply four approaches to compare the accuracy of model and determine the optimum model to predict user churn. Four machine learning approaches are logistic regression, random forest, decision tree and gradient boosted tree (GBT).

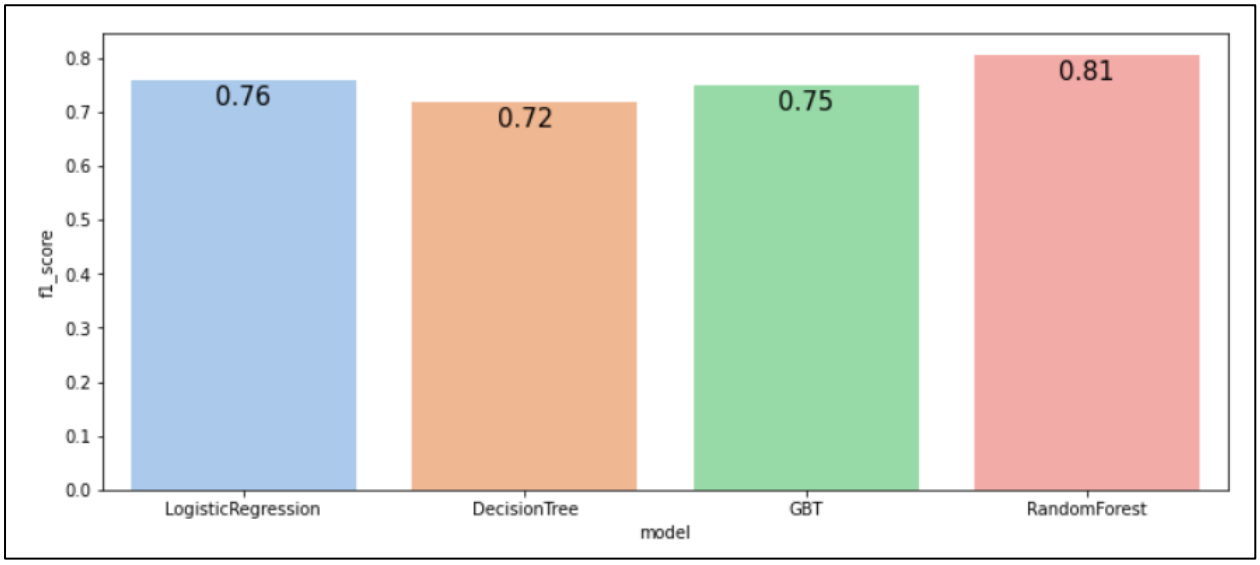
1.7 Method- Model Evaluation

By training the data with different classifier, we want to conclude which model provides best prediction on user churn. Therefore, we evaluated the four models based on their accuracy and f1 score. Accuracy is the percentage of correct predictions for the test data, the value of accuracy is between 0 to 1, the closer the value to 1 indicating the better accuracy. On the other hand, f1 score is a machine learning evaluation metric that measures a model's accuracy. It combines the precision and recall scores of a model. The accuracy metric computes how many times a model made a correct prediction across the entire dataset. F1 score is usually more useful than

accuracy, especially when there is uneven class distribution. The value of f1 score is between 0 to 1, the closer the value to 1 indicating the better model is working.

Based on the calculation, we observe that random forest model obtains the highest accuracy and f1 score of 0.81 as compared to decision tree, GBT and logistic regression.





Coursework 2 Component 3: Summary & Conclusion

2. Conclusion

In summary, we able to study the Sparkify dataset on the user churn topic and created functions for the modelling process. The dataset possesses sufficient quality with predictive features. In this project, we trained four model to find an optimum model to predict churn for future deployment. The model that we trained included logistic regression, decision tree, GBT and random forest.

The main challenges and complication we found was that the feature selection of the dataset was not trivial. The choice of the features has limitation in power and memory. Therefore, we use the subset of the dataset to obtain good understanding of the Sparkify users' behavior. We able to identify the user churn and transform the raw data into a meaningful binary classification predictive model.

The best model we observed from our work is the random forest model with both the highest accuracy and f1 score as compared to 0.7 accuracy in decision tree, 0.73 accuracy in GBT and 0.75 accuracy in logistic regression. Overall, we able to deploy HDFS, PySpark and MLLib to apply machine learning technique on model training for user churn. This approach can also be applied similarly on other dataset to study on user churn on any subscription services.

In the future, to improve the model, we may need more data on user behavior to understand the reason of user churn. We may include a study on sentiment analysis to study the impact of emotional aspect on the churn of user as the user may churn due to many reasons for example, they do not find any songs that they like or the Spakify platform is not user friendly instead of just making a conclusion that the user churn due to subscription fee and hence provide more discount on the fee.

3. Total number of words

Word counts excluding cover page, codes, figure, and references: 1965 words

4. References

1. Ahmad, A. K. (2019). Customer churn prediction in telecom using machine learning in big data platform. Retrieved from https://journalofbigdata.springeropen.com/articles/10.1186/s40537-019-0191-6
2. Almuqren, L. (2021). An Empirical Study on Customer Churn Behaviours Prediction Using Arabic Twitter Mining Approach. Retrieved from https://www.mdpi.com/1999-5903/13/7/175
3. Cochet, C. (2021). Predicting customer churn for a popular music streaming service. Retrieved from https://github.com/ChristopherCochet/sparkify-customer-churn-prediction
4. Forhad, N. (2014). Churn analysis: Predicting churners. Retrieved from https://www.researchgate.net/publication/295257947_Churn_analysis_Predicting_churners
5. Prabadevi, B. (2023). Customer churning analysis using machine learning algorithms. Retrieved from https://www.sciencedirect.com/science/article/pii/S2666603023000143
6. Tozatto, T. (2022). Churn Prediction for Music Streaming App — Sparkify. Retrieved from https://blog.devgenius.io/churn-prediction-for-music-streaming-app-sparkify-d6e26d1ac80f